

Informe Tarea 1

Daniel Parra Torres, dparrat@usm.cl

Vicente San Martín, vsanmartinf@usm.cl

Benjamín Olguín Olivares, bolguin@usm.cl

Vicente Púa Riquelme, vpua@usm.cl

Krishna Hidalgo, khidalgo@usm.cl

1. Parte 1

1.1. Diseño de la función “lecturaSensor()”

La función “lecturaSensor()” fue diseñada con el objetivo de simular la lectura de un sensor de tipo electrónico. Para lograrlo, la función genera un valor aleatorio dentro de un rango determinado, representando así una medición obtenida desde un sensor físico, que en este caso es hipotético.

La implementación utilizada fue la siguiente:

```
double lecturaSensor() {  
    double lecturaAleatoria;  
    lecturaAleatoria = rand() % 256;  
    return lecturaAleatoria;  
}
```

1.1.1. Requisitos operacionales de la función

La función debe cumplir con los siguientes requisitos:

- Simular el comportamiento de un sensor real.
- Entregar un valor numérico de lectura.
- Generar valores dentro de un rango acotado dado.
- Permitir múltiples lecturas consecutivas para distintos sensores.
- Integrarse fácilmente al sistema principal mediante modular esta tarea.

Para cumplir estos requisitos, se utilizó la función “rand()” de la biblioteca de funciones estándar de C, la cual permite generar números pseudoaleatorios. Después, el operador módulo “% 256” restringe los valores tomados del intervalo entre 0 y 255, simulando lecturas en mili volts [mV].

La función retorna un valor de tipo “double”, permitiendo representar lecturas numéricas provenientes de sensores, aunque posteriormente el valor es convertido a entero al almacenarse en el arreglo de sensores.

1.1.2. Justificación del uso del “enum Accion”

El programa implementa un nuevo tipo de dato denominado “Accion”:

```
enum Accion {  
    INIT,  
    READ,  
    SHOW  
};
```

El uso de “enum” permite representar de manera clara las distintas operaciones que puede realizar el sistema de los sensores.

Los elementos a continuación representan una acción específica:

- INIT: Inicializa los sensores en cero.
- READ: Realiza la lectura de los sensores.
- SHOW: Muestra los valores almacenados.

La utilización de “enum” mejora considerablemente la legibilidad y organización del código, ya que evita usar números arbitrarios para identificar acciones, y lo hace un poco más realista. Además, facilita la mantención y escalabilidad del programa, permitiendo agregar nuevas acciones en el futuro sin modificar la estructura principal, siendo que la solución que se implemento fue modular.

1.1.3. Funcionamiento general de la función “usarSensores()”

La función “usarSensores()” recibe dos parámetros:

```
void usarSensores(int sensores[], enum Accion acc)
```

- Un arreglo de sensores.
- Una acción del tipo “Accion”.

Dependiendo de la acción recibida, la función ejecuta distintas tareas utilizando una estructura “switch”, por ende, por casos.

Como se puede ver a continuación:

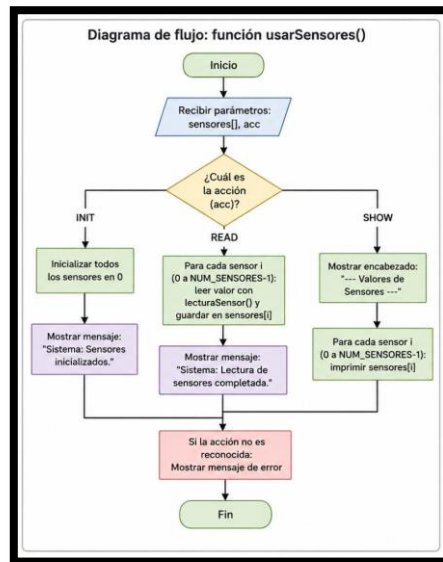


Diagrama 1: Función “usarSensores()”

1.2. Diseño completo de la nueva función “main()”

La función “main()” corresponde al punto de entrada principal del programa. Su función es coordinar el funcionamiento total del sistema de sensores utilizando la función modular creada “usarSensores()”.

La implementación fue la siguiente:

```
int main(int argc, char* argv[]) {  
    int sensores[NUM_SENSORES];  
    usarSensores(sensores, INIT);  
    usarSensores(sensores, READ);  
    usarSensores(sensores, SHOW);  
    exit(EXIT_SUCCESS);  
}
```

1.3 Funcionamiento de la función “main()”

El programa realiza las siguientes etapas:

Creación del arreglo de sensores

```
int sensores[NUM_SENSORES];
```

Se crea un arreglo capaz de almacenar las lecturas de todos los sensores definidos por la constante “NUM_SENSORES”.

Inicialización de sensores

```
usarSensores(sensores, INIT);
```

Se llama a la función modular indicando la acción “INIT”, lo que provoca que todos los sensores se inicialicen en cero, usando ese caso.

Lectura de sensores

```
usarSensores(sensores, READ);
```

El programa ejecuta la lectura simulada de cada sensor, extrayendo los datos para posterior entrega, utilizando la función “lecturaSensor()”.

Cada valor generado se almacena dentro del arreglo.

Visualización de resultados

```
usarSensores(sensores, SHOW);
```

Finalmente, se muestran en pantalla los valores almacenados en cada sensor.

Finalización del programa

```
exit(EXIT_SUCCESS);
```

El programa termina correctamente indicando ejecución exitosa, y finalizando esta parte.

1.3. Ventajas del diseño modular implementado

La nueva implementación modular presenta múltiples ventajas:

- Mejora la organización del código.
- Facilita la reutilización de funciones.
- Reduce la duplicación de instrucciones.
- Simplifica la corrección y mantención.
- Permite ampliar el sistema fácilmente.

Además, el uso de “enum” hace que las acciones del sistema sean más comprensibles y seguras, es decir hace que sea un código más cohesivo y lo más independiente del resto del código.

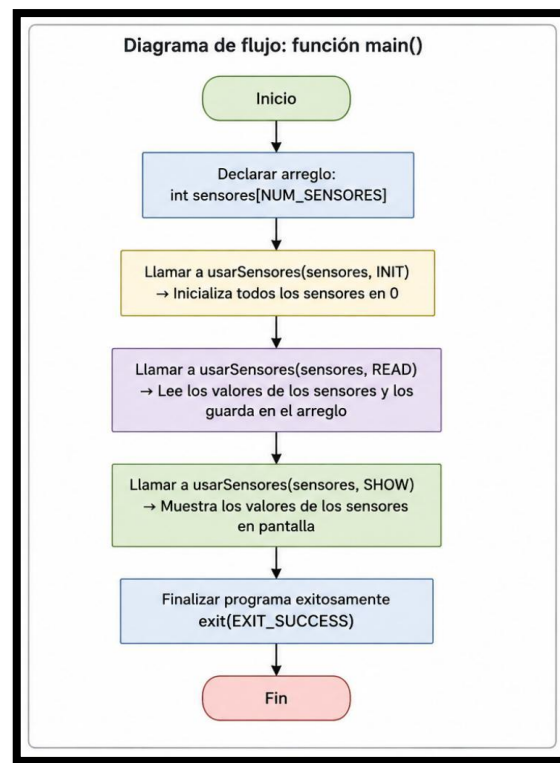


Diagrama 2: Función “main()”

2. Parte 2

2.1. Análisis de la función “cicloEjemplo”

La función “cicloEjemplo” se encarga de retornar el tiempo promedio que necesita el ordenador para realizar una acción en específico en nanosegundos, en este caso está midiendo el tiempo que tarda en escribir el símbolo “*” un millón de veces

2.1.1. ¿Qué es lo que retorna la función clock()?

La función “clock()” retorna un valor del tipo “clock_t” (el cual puede ser un int o un double) y que representa el tiempo en “ticks” de reloj usados por un programa, importante resaltar que solo mide el tiempo efectivo del programa, de la misma forma en la que la jornada laboral no tiene en cuenta el tiempo de traslado al trabajo.

2.1.2 ¿Que unidades tienen las variables inicio y fin?

Las variables inicio y fin tienen unidades de “ticks de reloj” o “clock ticks” las cuales se guardan una variable de tipo uint64_t, es decir, un entero sin signo de 64 bits.

2.1.3 ¿Por qué para obtener el tiempo total de duración del ciclo “for” se debe dividir la diferencia entre “fin” e “inicio” con el valor de la constante “CLOCKS_PER_SEC”?

El objetivo de la función es retornar el tiempo que se demora un programa en ejecutarse, no nos interesa cantos “ticks” tarda, debido a esto y sabiendo que las variables inicio y fin están medidas en [tick] y que la constante “CLOCK_PER_SEC” tiene medidas [tick/s], al dividir las obtenemos un valor medido en [s].

2.1.4 Explicar que hace “(double)” como prefijo de “(fin – inicio)”

Este prefijo está transformando explícitamente el dato “(fin – inicio)” de uint64_t a un “double”.

2.1.5 Explicar para que sirve usar “(double)” como prefijo de “(fin – inicio)”

El objetivo de transformar el resultado de la resta a un double es que, si no se hiciera, la división entre 2 números enteros dará como resultado otro número entero el cual no tendrá en cuenta los decimales de la división, para evitar eso se transforma un dato a double con el objetivo de que el resultado también sea un double y de esta forma evitando la pérdida de decimales.

2.2. Diseño de la función “cicloPrueba(caso_t caso)”

La función “cicloPrueba(caso_t caso)” fue diseñada con el objetivo de obtener de manera precisa el tiempo promedio en el que demora el equipo en realizar una tarea específica, realizando 100 millones de iteraciones y obteniendo el tiempo promedio que se demora en realizarlas.

Para esto se implementó la siguiente función:

```
double cicloPrueba(caso_t caso){
    double tiempoPromedio;
    double cantidadIteraciones;
    uint64_t inicio, fin;
    tiempoPromedio = 0.0;
    cantidadIteraciones = 1e8;

    inicio = clock();
    switch (caso)
    {
    case VACIO:
        for(int64_t i=0; i<cantidadIteraciones; i++){
            ;
        }
        break;
    case ESPACIO:
        for(int64_t i=0; i<cantidadIteraciones; i++){
            printf(" ");
        }
        break;
    case ASTERISCO:
        for(int64_t i=0; i<cantidadIteraciones; i++){
```

```
        printf("*");
    }
    break;
case NEWLINE:
    for(int64_t i=0; i<cantidadIteraciones; i++){
        printf("\n");
    }
    break;
default:
    break;
}
fin = clock();
tiempoTotal = (((double) (fin - inicio)) / CLOCKS_PER_SEC) * 1e9;
double tiempoPromedio = tiempoTotal / cantidadIteraciones;
return tiempoPromedio;
```

2.2.1 Requisitos operacionales de la función

- Realice una gran cantidad de iteraciones
- Discriminar entre los 4 casos posibles
- Retornar en segundos el tiempo que tarda en realizar la actividad

2.2.2 Justificación del switch

Para la realización de esta función se tomó como base la función "cicloEjemplo()", la cual se le agrego un switch que le permite hacer en una misma función tomar mediciones del tiempo que demora el sistema en realizar 4 acciones distintas, ya sea mostrar " ", un "*", hacer un cambio de línea o simplemente hacer un bucle vacío.

2.2.3 ¿Que hace “cicloPrueba()”?

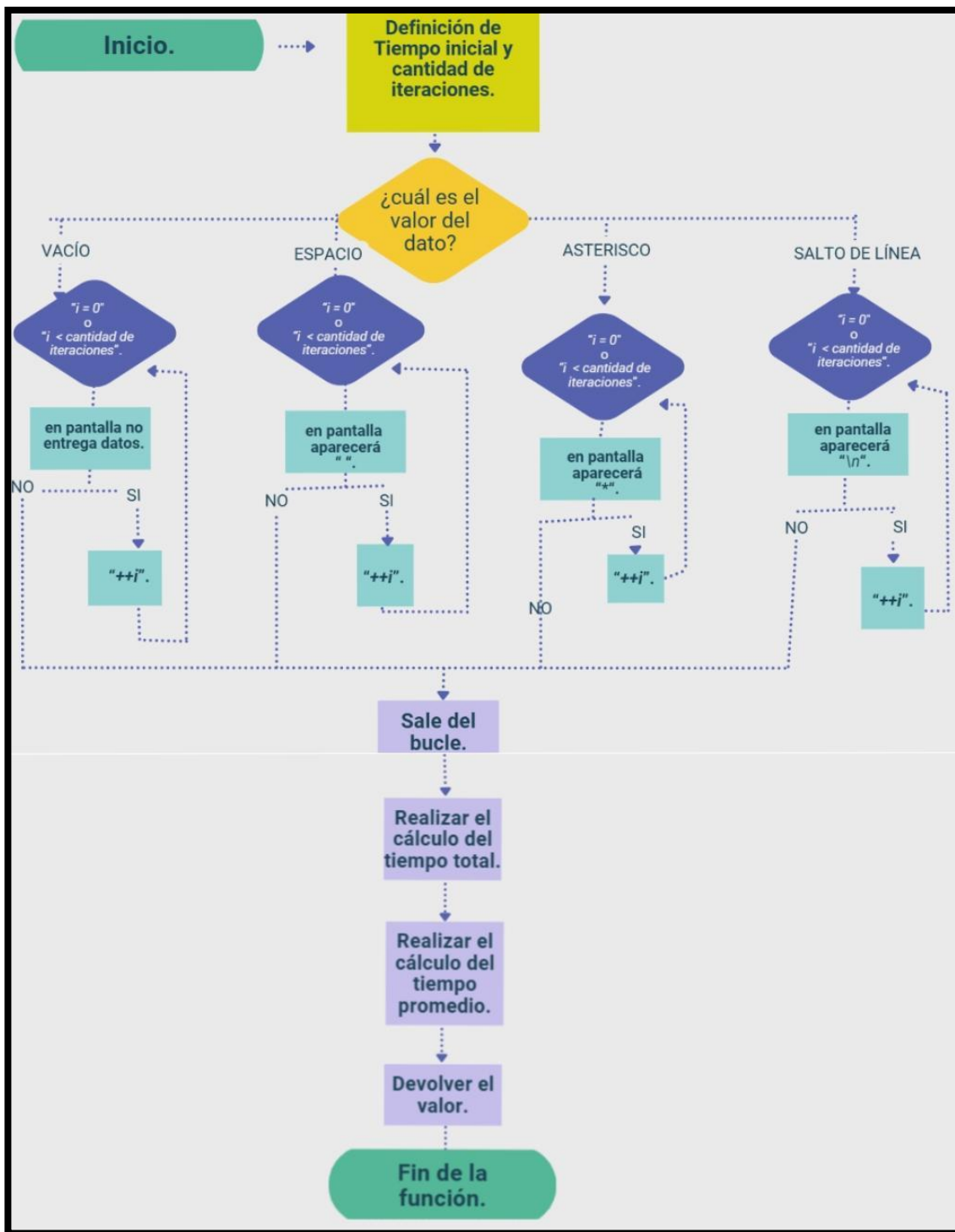


Diagrama 3: Función “cicloPrueba()”

Esta función en primer lugar almacena la cantidad de “ticks” consumidos por la CPU hasta el momento en la variable inicio. Luego dependiendo del caso que le pedimos repite 100 millones de veces la acción solicitada, una vez hechas guarda la cantidad de “ticks” consumidos en la variable fin.

Para obtener el tiempo total de se demora la CPU primero trasformamos explícitamente la resta entre “inicio” y “fin” para dividirla por la constante “CLOCKS_PER_SEC” la cual es una constante que simboliza la cantidad de “ticks” por segundo.

Podemos notar que el resultado de la división es un numero demasiado pequeño, por lo tanto, podemos multiplicarlo por 10^9 para obtener los nanosegundos que tarda en realizar la acción.

Finalmente dividimos el tiempo total por la cantidad de iteraciones y de esta forma obtendremos finalmente el tiempo promedio el cual es retornado por la función.

2.3. Análisis Final de Datos

Para conseguir los tiempos de ejecución, se ajustó el código para realizar 100.000.000 iteraciones ($1e8$) por cada caso. El programa se compiló utilizando el archivo “Makefile” brindado y se ejecutó desde la terminal de PowerShell en Visual Studio Code. Ahora bien, se realizaron un total de 10 ejecuciones independientes del programa “mediciones.exe” para así conseguir las muestras necesitadas.

Las especificaciones del equipo en que se realizaron las pruebas fueron las siguientes:

- Sistema Operativo: Windows 11 Home Single Language (64 bits)
- Procesador: 12th Gen Intel(R) Core(TM) i5-1235U @ 1.30 GHz
- Memoria RAM: 16.0 GB
- Compilador: g++ usando el estándar C++17

A continuación, se muestran los tiempos obtenidos luego de las 10 ejecuciones para cada uno de los casos evaluados, junto con sus respectivos promedios y desviaciones estándar. Los valores están expresados en nanosegundos [ns].

Caso	tprom 1	tprom 2	tprom 3	tprom 4	tprom 5	tprom 6	tprom 7	tprom 8	tprom 9	tprom 10	Promedio	Desv. Estándar
VACIO	0	0	0	0	0	0	0	0	0	0	0	0
ESPACIO	15100	23050	28250	20220	20860	16540	22900	24160	22610	21500	21519	3739,261514
ASTERISCO	15090	20590	24400	19920	20720	17060	22700	22130	22800	20580	20599	2776,886546
NEWLINE	33750	37080	41790	33890	35630	34360	37130	30590	34340	33160	35172	3010,019564

Tabla 1: Tiempo promedio en nanosegundos [ns] obtenido tras 10 ejecuciones independientes (casos VACÍO, ESPACIO, ASTERISCO y NEWLINE)

Análisis de resultados:

En base a los datos conseguidos, se logra evidenciar una diferencia gigante de rendimiento entre los diferentes casos, lo que demuestra el gran costo computacional de utilizar un sistema de Entrada/Salida (I/O), en comparación con el uso exclusivo de la memoria y procesador del sistema:

- Caso VACIO (Ciclo vacío): El tiempo de ejecución es nulo o prácticamente nulo (0 [ns] en las 10 mediciones). Esto es debido a que el ciclo vacío procesa directamente en los registros de la CPU y memoria caché, sin tener la necesidad de interactuar con periféricos externos. Esta se podría catalogar como una operación a nivel de hardware extremadamente rápida.
- Caso ESPACIO y ASTERISCO: Los tiempos rondan un promedio cercano a los 20000 – 21000 [ns]. Esto se debe principalmente al uso de la función “*printf*”, ya que, al imprimir un carácter, el programa detiene su flujo normal para realizar una llamada al sistema operativo (system call) y transferir datos al hardware para renderizar el texto en la consola. Al fin y al cabo, todo este proceso I/O genera un gran cuello de botella mucho más grande en comparación con el cálculo en sí.
- Caso NEWLINE: Este es el caso más lento de los 4 con un promedio 35172 [ns]. Este presenta un gran costo de la operación I/O, además de imprimir un salto de línea “\n”, que hace que sea necesario actualizar y refrescar la pantalla, provocando una latencia adicional al proceso.